Quick start: from a validated model to an interactive page

This guide is for someone who wants a result, not the internals. The manual has the details. A PDF copy of this guide is in `docs/pdf/QUICKSTART.pdf`.

A few words used below:

- **Stock**: something that accumulates, such as customers or firms in evaluation.
- **Flow**: a rate that fills or drains a stock, such as deals won per month.
- **Loop**: a chain of causes that comes back to where it started. **R** (reinforcing) loops amplify change; **B** (balancing) loops push back.
- **Lever**: an input you can move on the page, within the range the model was validated on.
- **Baseline**: the run every result is compared with.

Prerequisites

- **Python 3.11 or 3.12** (PySD may not support newer). From the repository root:

  ```
  python3.12 -m venv .venv && .venv/bin/pip install -r engine/requirements.txt
  ```

- **node.** The parity gate runs the page's engine under node on every build and never skips: without node, the build and every test that builds a page fail.
- **Playwright with Chromium**, for the browser tests. Install Playwright in a folder that node resolves from the repository (for example its parent folder), install Chromium, then check from the repository root that node can load it:

  ```
  (cd .. && npm install --no-save playwright)
  ```

  ```
  (cd .. && npx playwright install chromium)
  ```

  ```
  node -e "require('playwright')"
  ```

  The check must finish without an error. Without Playwright the browser tests are skipped, not failed: a recorded run of the suite gave 671 passed and 237 skipped (0 failed). A release run adds `--require-browser` (or sets `SD_REQUIRE_BROWSER=1`), so a missing browser fails the run instead of being skipped.

Install once, use anywhere

The two skills need to know where the engine is. Inside an unpacked copy, either run the installer once (below) or export `SD_FORGE_HOME=$PWD` (quoted if the path has spaces) after building this folder's `.venv`; without one of them the skill stops with "sd-forge is not installed". Installing also lets you use `/sd-model` and `/sd-explorer` from any folder. The installer does the whole setup: it builds the engine's Python environment

itself, checks that node is present, and links the skills into Claude Code. You only need git (advanced route) or a web browser (simple route).

Simple route. On the repository page choose Code, then Download ZIP. Unpack it, open the unpacked folder and double-click `setup.command` (the first time, macOS may refuse to open a downloaded file: right-click it and choose Open). Or run it in a terminal from that folder:

```
bash setup.command
```

`setup.command` is the one executable for the whole setup. It first checks that a usable Python exists, so nothing is copied or changed if there is none. If the folder is in Downloads, on the Desktop or in a temporary place, it offers to copy it to a folder called sd-forge in your home folder, because the skills read the engine from the folder they were installed from and that folder must not be deleted or moved later; the original is left alone and can be deleted afterwards. It then runs the installer described below. Options it understands are listed in the comments at its top; everything else goes to the installer.

Advanced route. Clone instead of downloading, then install the same way:

```
git clone https://github.com/fourpoint669/sd-forge.git "$HOME/sd-forge"
cd "$HOME/sd-forge"
bash install-skills.sh
```

What the installer does, in order:

1. It looks for Python 3.11 or 3.12 (newest first) and uses one of those if it can. If the machine has only a newer Python, it builds the environment with it and then proves it works: it validates the shipped example project and requires the result files to equal the committed ones. If they differ, it deletes that environment and refuses that Python. If you have no usable Python, it stops before changing anything and says how to get one.
2. It builds the environment in the `.venv` folder from the pinned package list, which needs the internet and about a minute. An environment that already works is left alone.
3. It warns if node is missing, because the explorer build needs it.
4. It records where the folder is in `~/.config/sd-forge/home` and links the two skills and the two slash commands into `~/.claude/skills` and `~/.claude/commands` (Claude Code). If Codex is installed (its `~/.codex` folder, or `CODEX_HOME`, or the `codex` command), it also links the two skills into `~/.codex/skills`; two options of the installer force or skip that (they are listed in the comments at the top of `install-skills.sh`). It refuses to overwrite a file or folder that is not its own link, and changes nothing in `~/.claude` if any step fails.
5. It prints, and on a Mac shows in a dialog box, what to type next in each tool it installed for.

It is safe to run again. If you move the folder, run the installer again from its new location; it tells you how to re-point the old links. `uninstall-skills.sh` removes the links and the home file, and leaves the `.venv` folder alone.

After that, start Claude Code or Codex in any folder (a session that was already open needs restarting). Claude Code uses a slash, Codex a dollar sign; the skills are the same files:

```
/sd-model "<your question>"
$sd-model "<your question>"
```

Windows. Not tested, and not supported: the installer is a shell script and the skills call the interpreter in the environment's `bin` folder. If you want to try anyway, use WSL (Windows Subsystem for Linux, which is Linux and the closer match) or Git Bash, and ask Claude Code or Codex to clone the repository, build the `.venv` and set `SD_FORGE_HOME`. On native Windows the environment's interpreter is in its `Scripts` folder, not `bin`, so the agent must use that one wherever the skills use the `bin` one. Keep the repository's `.gitattributes`: it stops git from converting line endings, which would change file hashes and make the release gate treat the model files as edited after validation.

ChatGPT. The ChatGPT website and phone apps cannot use these skills: they have no access to your computer, and the skills run Python and node on it. Codex signs in with a ChatGPT account, so with one you can use the skills there.

What each does:

- `/sd-model "<your question>"` writes the model to a new folder `<model-name>/` in the folder you are in, and `/sd-explorer <model-name>` writes `explorer.html` into that model's folder. Nothing is written into the repository.
- To keep models in one place, set `SD_PROJECTS_DIR` to that folder before starting Claude Code. The same setting is the `--projects-root` option of the engine scripts; the option wins over the setting, and the setting over the current folder.
- `SD_FORGE_HOME` names the repository and wins over the file the installer wrote. If neither exists, or the folder has no `engine/validate.py`, the skills stop and say what to fix.
- Every check, refusal and run mode is the same as inside the repository. A model that depends on another one (as `saas-churn-vs-winrate` does on `bass-b2b-adoption`) needs that model in the same folder.

The same build from a plain terminal, from any folder:

```
"$SD_FORGE_HOME/.venv/bin/python" -I "$SD_FORGE_HOME/engine/build_explorer.py" --projects-root ~/models my-model
```

What to type

In Claude Code, inside this repository (or, once installed, in any folder, naming the model folder):

```
/sd-explorer projects/saas-churn-vs-winrate
```

Name a project folder, or name none to be offered every validated project. To build a new model first, use `/sd-model "<your question>"`, or `/sd-model --mode express "<your question>"` for a run with no pauses (`--mode full` pauses at every gate; the default, `checkpoint`, pauses after gate 5 and at the end). Its last gate (gate 10) runs `/sd-explorer` for you. A checkpoint or express model says on its summary and on its page that a person did not review it.

Models are code: validating or building one runs it on your computer, so only use models you trust.

Without Claude Code, the same build is one command:

```
.venv/bin/python -I engine/build_explorer.py projects/saas-churn-vs-winrate
```

A project you made yourself needs its delay audit first, once and again after any model change: `.venv/bin/python -I engine/delay_audit.py projects/<m>`. The shipped projects already have theirs. Without it the build stops with a file-not-found error for `delay_audit.json`.

What you will be asked

One round of 4 multiple-choice questions, all at once. Every question can be skipped; a skipped question takes its default. If nobody can answer (an unattended run), every default is used and the build does not wait.

- **Which project page(s) should be built?** Default: Only saas-churn-vs-winrate. Other choices: All validated projects, Only bass-b2b-adoption instead, Only car-dealership-profit instead.
- **Which decision metric and baseline should the page lead with?** Default: ARR at month 60 vs no programme. Other choices: Cumulative revenue at month 60 vs no programme, Cumulative revenue net of programme spend at month 60 vs no programme, ARR at month 60 vs churn first (months 0-24 churn, 24-60 win rate).
- **Which levers should move on the page?** Default: All levers. Other choices: Policy levers only, Policy levers and estimated parameters.
- **Who is the page mainly for?** Default: Decision makers. Other choices: Modellers, Workshop.

The answers change only what the page shows first. They never change the model, its ranges or the checks. Every answer, and every default that was assumed, is listed in the page's **Model limits** window.

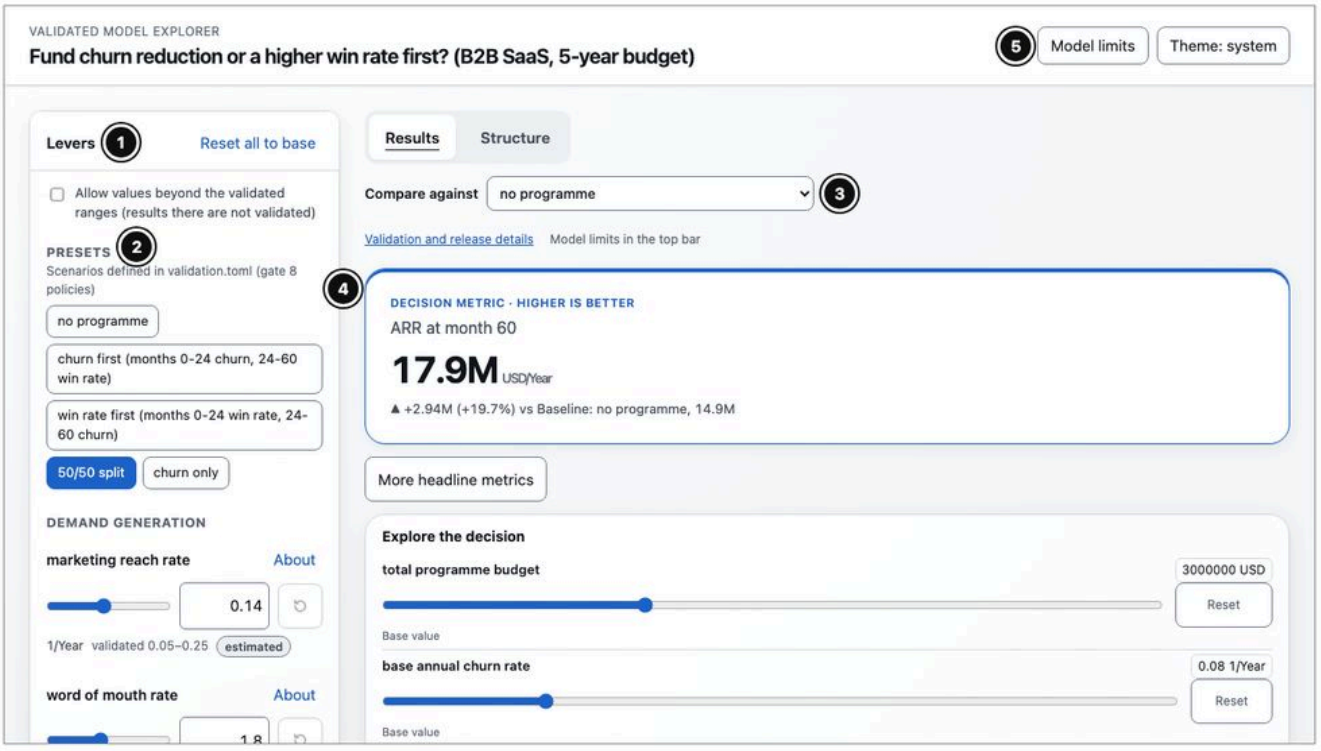
What you get

`projects/<m>/explorer.html`: one file, about 497 KB for `saas-churn-vs-winrate`. It needs no internet connection or install. Open it in a browser or send it as an attachment.

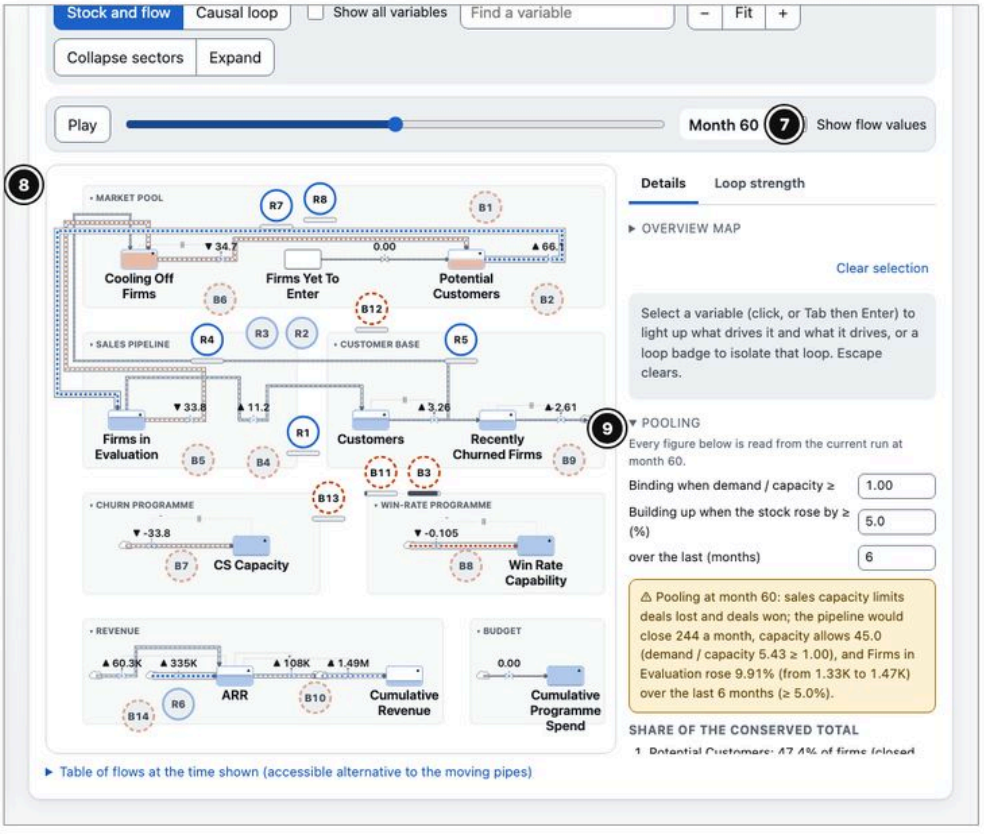
Next to it, the build writes `explorer_parity.json` (the evidence that the page matches the validated model) and records your answers in `explorer_intake.toml`.

How to read the page

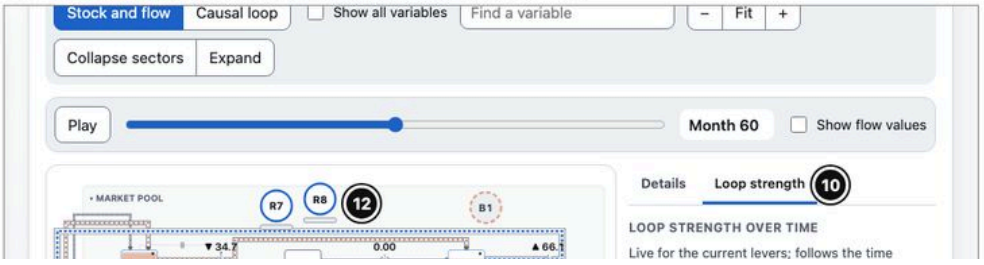
A. Results tab (markers 1-6)

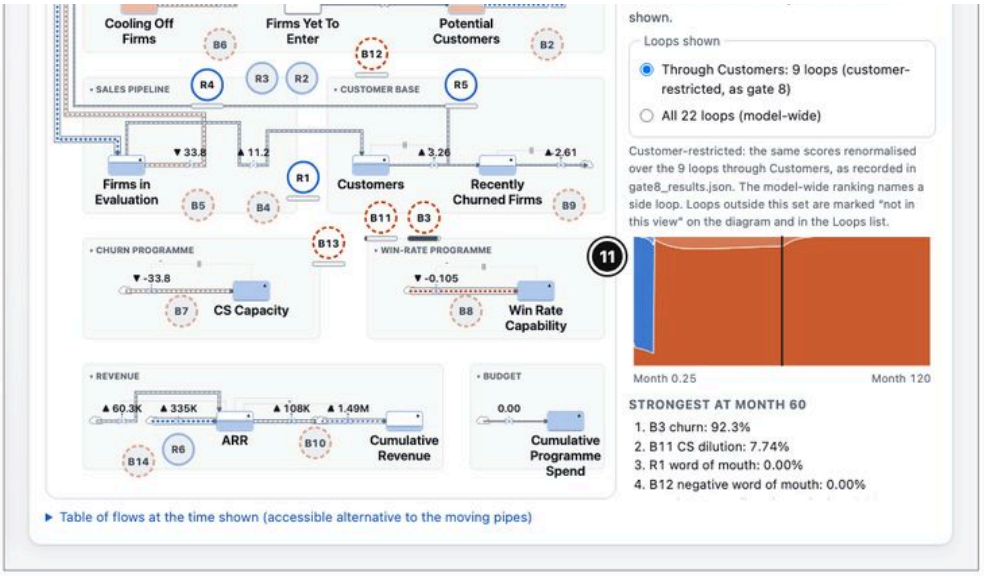


B. Structure tab, Details side tab (markers 7-9)



C. Structure tab, Loop strength side tab (markers 10-12)





The screenshot is taken from the real build of `saas-churn-vs-winrate`. The numbers below match the markers.

Results tab

1. **Levers.** Each input has a slider and a box, limited to its validated range. "About" shows its unit, definition, source and confidence tag (`verified`, `reported` or `estimated`). Ticking "Allow values beyond the validated ranges" shows a warning banner, and every result computed outside the ranges is labelled "outside validated range".
2. **Presets.** The scenarios defined when the model was validated. One click sets every lever.
3. **Compare against.** The baseline. Every difference on the page (▲ or ▼, and the dashed line in the charts) is measured against it.
4. **Headline numbers.** The decision metric comes first and is marked. Each card gives the current value and the change from the baseline. These are single runs, not forecasts.
5. **Model limits.** What this model cannot tell you, taken word for word from the project's `summary.md`, plus the page settings from the intake. It is always one click away.
6. **Compare scenarios.** Save up to 4 lever settings. The table compares them with the baseline, and each chart overlays them.

Below these are the charts. A shaded band appears only where a Monte Carlo run applies to the line it surrounds (a run of many samples drawn across the parameter ranges). A grey, hatched band belongs to the baseline, not to your current setting. A single run has no band of its own.

Structure tab, Details

7. **Time bar.** Play, or drag the slider, to move through time. The diagram, the flows table and the side panel follow the time shown, and so does the crosshair in the Results charts.
8. **The diagram.** Stocks are tanks, flows are pipes with valves, and loops are the lettered badges. Pipe width follows the flow at the time shown; blue with ▲ means higher than the baseline, orange with ▼ lower. Select a variable to follow its chain, or a badge to isolate its loop. "Causal loop" switches to a cause-and-effect view; "Show all variables" adds every auxiliary.
9. **Pooling.** Where a capacity limit is active, this panel says so, using figures from the current run. It flags a limit when demand ÷ capacity is at least 1, and flags a build-up when the waiting stock rose by at least 5% over the last 6 months. You can change both thresholds. A model with no capacity-limited flow says that pooling does not apply.

Structure tab, Loop strength

10. **Loop strength tab.** How strongly each loop drives the behaviour at the time shown, computed for your current levers (see "loops that matter" in the manual's glossary).
11. **Strength over time.** Each loop's share of strength, stacked over the whole run, with the dominant loop by period listed below. For `saas-churn-vs-winrate` the default view is the loops through Customers, as in the project's policy analysis; you can switch to all loops.
12. **Badge bar.** The bar under a loop badge is that loop's share among the loops shown. A faded badge with no bar is "not in this view": it is outside the selected set and has no share there (not a zero share).

On a phone, the page switches to three tabs: Levers, Results and Structure.

If the build refuses

The build refuses rather than publish a page it cannot stand behind. Its message starts with `explorer withheld:` and names the reason.

What the message names	What to do
`validation_report.md` or `validation_results.json` not passing	Run validation and fix what fails: `.venv/bin/python -I engine/validate.py projects/<m>/validation.toml`.
a model file that "changed after validation"	The `.mdl` or `.py` was edited after validation. Re-run `engine/validate.py`, and `checks.py` if the project has one.
`checks_results.json`	Re-run the project's checks, for example `.venv/bin/python -I projects/saas-churn-vs-winrate/checks.py`.
a variable with no definition	Add a plain-language entry to `projects/<m>/explorer_glossary.toml`.
`parity gate failed`	The browser engine does not match the validated model. Do not adjust anything to make it pass; report it.

The manual lists every refusal with its exact message.

Three common mistakes

1. **Reading a single run as a forecast.** The headline numbers and the line in each chart are one run at the levers you set. Uncertainty is shown only by the Monte Carlo bands, and only where they apply.
2. **Pushing levers beyond the validated range and trusting the result.** The page lets you, but it labels every such result "outside validated range": the model was not validated there.
3. **Editing the model or its results to get past a refusal.** The build compares the model files with the ones validation recorded. Re-run validation instead. Never edit `explorer.html` by hand either: rebuild it.

Key takeaways

- One command or one slash command gives you one offline file. Skip any question you don't care about.
- Read the headline cards against the baseline you chose, and read Model limits before quoting a number.
- If the build refuses, fix the project, not the page.

The explorer keeps the decision result on an opaque surface. Primary controls provide shortcuts to existing levers; the complete Levers panel retains their definitions, sources and validated ranges. On smaller screens, use More headline metrics for supporting results. Inspect all baseline and scenario metrics opens the comparison table, and saving a scenario opens it automatically. Validation and release details links to the recorded provenance. Glass effects are limited to navigation chrome and fall back to solid surfaces.